

Defining Repository Interfaces

Defining Repository Interfaces

Fine-tuning Repository Definition

Using Repositories with Multiple Spring Data Modules

To define a repository interface, you first need to define a domain class-specific repository interface. The interface must extend `Repository` and be typed to the domain class and an ID type. If you want to expose CRUD methods for that domain type, you may extend `CrudRepository`, or one of its variants instead of `Repository`.

Fine-tuning Repository Definition

There are a few variants how you can get started with your repository interface.

The typical approach is to extend `CrudRepository`, which gives you methods for CRUD functionality. CRUD stands for Create, Read, Update, Delete. With version 3.0 we also introduced `ListCrudRepository` which is very similar to the `CrudRepository` but for those methods that return multiple entities it returns a `List` instead of an `Iterable` which you might find easier to use.

If you are using a reactive store you might choose `ReactiveCrudRepository`, or `RxJava3CrudRepository` depending on which reactive framework you are using.

If you are using Kotlin you might pick `CoroutineCrudRepository` which utilizes Kotlin's coroutines.

Additionally you can extend `PagingAndSortingRepository`, `ReactiveSortingRepository`, `RxJava3SortingRepository`, or `CoroutineSortingRepository` if you need methods that allow to specify a `Sort` abstraction or in the first case a `Pageable` abstraction. Note that the various sorting repositories no longer extended their respective CRUD repository as they did in Spring Data Versions pre 3.0. Therefore, you need to extend both interfaces if you want functionality of both.

If you do not want to extend Spring Data interfaces, you can also annotate your repository interface with `@RepositoryDefinition`. Extending one of the CRUD repository interfaces exposes a complete set of methods to manipulate your entities. If you prefer to be selective about the methods being exposed,

copy the methods you want to expose from the CRUD repository into your domain repository. When doing so, you may change the return type of methods. Spring Data will honor the return type if possible. For example, for methods returning multiple entities you may choose `Iterable<T>`, `List<T>`, `Collection<T>` or a VAVR list.

If many repositories in your application should have the same set of methods you can define your own base interface to inherit from. Such an interface must be annotated with `@NoRepositoryBean`. This prevents Spring Data to try to create an instance of it directly and failing because it can't determine the entity for that repository, since it still contains a generic type variable.

The following example shows how to selectively expose CRUD methods (`findById` and `save`, in this case):

Selectively exposing CRUD methods

JAVA

```
@NoRepositoryBean
interface MyBaseRepository<T, ID> extends Repository<T, ID> {

    Optional<T> findById(ID id);

    <S extends T> S save(S entity);
}

interface UserRepository extends MyBaseRepository<User, Long> {
    User findByEmailAddress(EmailAddress emailAddress);
}
```

In the prior example, you defined a common base interface for all your domain repositories and exposed `findById(...)` as well as `save(...)`. These methods are routed into the base repository implementation of the store of your choice provided by Spring Data (for example, if you use JPA, the implementation is `SimpleJpaRepository`), because they match the method signatures in `CrudRepository`. So the `UserRepository` can now save users, find individual users by ID, and trigger a query to find `Users` by email address.

NOTE

The intermediate repository interface is annotated with `@NoRepositoryBean`. Make sure you add that annotation to all repository interfaces for which Spring Data should not create instances at runtime.

Using Repositories with Multiple Spring Data Modules

Using a unique Spring Data module in your application makes things simple, because all repository interfaces in the defined scope are bound to the Spring Data module. Sometimes, applications require us-

ing more than one Spring Data module. In such cases, a repository definition must distinguish between persistence technologies. When it detects multiple repository factories on the class path, Spring Data enters strict repository configuration mode. Strict configuration uses details on the repository or the domain class to decide about Spring Data module binding for a repository definition:

1. If the repository definition extends the module-specific repository, it is a valid candidate for the particular Spring Data module.
2. If the domain class is annotated with the module-specific type annotation, it is a valid candidate for the particular Spring Data module. Spring Data modules accept either third-party annotations (such as JPA's `@Entity`) or provide their own annotations (such as `@Document` for Spring Data MongoDB and Spring Data Elasticsearch).

The following example shows a repository that uses module-specific interfaces (JPA in this case):

Example 1. Repository definitions using module-specific interfaces

```
interface MyRepository extends JpaRepository<User, Long> { }

@NoRepositoryBean
interface MyBaseRepository<T, ID> extends JpaRepository<T, ID> { ... }

interface UserRepository extends MyBaseRepository<User, Long> { ... }
```

JAVA

`MyRepository` and `UserRepository` extend `JpaRepository` in their type hierarchy. They are valid candidates for the Spring Data JPA module.

The following example shows a repository that uses generic interfaces:

Example 2. Repository definitions using generic interfaces

```
interface AmbiguousRepository extends Repository<User, Long> { ... }

@NoRepositoryBean
interface MyBaseRepository<T, ID> extends CrudRepository<T, ID> { ... }

interface AmbiguousUserRepository extends MyBaseRepository<User, Long> { ... }
```

JAVA

`AmbiguousRepository` and `AmbiguousUserRepository` extend only `Repository` and `CrudRepository` in their type hierarchy. While this is fine when using a unique Spring Data module, multiple modules cannot distinguish to which particular Spring Data these repositories should be bound.

The following example shows a repository that uses domain classes with annotations:

Example 3. Repository definitions using domain classes with annotations

```
interface PersonRepository extends Repository<Person, Long> { ... }

@Entity
class Person { ... }

interface UserRepository extends Repository<User, Long> { ... }

@Document
class User { ... }
```

JAVA

`PersonRepository` references `Person`, which is annotated with the JPA `@Entity` annotation, so this repository clearly belongs to Spring Data JPA. `UserRepository` references `User`, which is annotated with Spring Data MongoDB's `@Document` annotation.

The following bad example shows a repository that uses domain classes with mixed annotations:

Example 4. Repository definitions using domain classes with mixed annotations

```
interface JpaPersonRepository extends Repository<Person, Long> { ... }

interface MongoDBPersonRepository extends Repository<Person, Long> { ... }

@Entity
@Document
class Person { ... }
```

JAVA

This example shows a domain class using both JPA and Spring Data MongoDB annotations. It defines two repositories, `JpaPersonRepository` and `MongoDBPersonRepository`. One is intended for JPA and the other for MongoDB usage. Spring Data is no longer able to tell the repositories apart, which leads to undefined behavior.

Repository type details and distinguishing domain class annotations are used for strict repository configuration to identify repository candidates for a particular Spring Data module. Using multiple persistence technology-specific annotations on the same domain type is possible and enables reuse of domain types across multiple persistence technologies. However, Spring Data can then no longer determine a unique module with which to bind the repository.

The last way to distinguish repositories is by scoping repository base packages. Base packages define the starting points for scanning for repository interface definitions, which implies having repository definitions located in the appropriate packages. By default, annotation-driven configuration uses the package of the configuration class. The base package in XML-based configuration is mandatory.

The following example shows annotation-driven configuration of base packages:

Annotation-driven configuration of base packages

JAVA

```
@EnableJpaRepositories(basePackages = "com.acme.repositories.jpa")
@EnableMongoRepositories(basePackages = "com.acme.repositories.mongo")
class Configuration { ... }
```



Copyright © 2005 - 2026 Broadcom. All Rights Reserved. The term "Broadcom" refers to Broadcom Inc. and/or its subsidiaries.

[Terms of Use](#) • [Privacy](#) • [Trademark Guidelines](#) • [Thank you](#) • [Your California Privacy Rights](#) • [Cookies Settings](#)

Apache®, Apache Tomcat®, Apache Kafka®, Apache Cassandra™, and Apache Geode™ are trademarks or registered trademarks of the Apache Software Foundation in the United States and/or other countries. Java™, Java™ SE, Java™ EE, and OpenJDK™ are trademarks of Oracle and/or its affiliates. Kubernetes® is a registered trademark of the Linux Foundation in the United States and other countries. Linux® is the registered trademark of Linus Torvalds in the United States and other countries. Windows® and Microsoft® Azure are registered trademarks of Microsoft Corporation. "AWS" and "Amazon Web Services" are trademarks or registered trademarks of Amazon.com Inc. or its affiliates. All other trademarks and copyrights are property of their respective owners and are only mentioned for informative purposes. Other names may be trademarks of their respective owners.